

以 WebAssembly Component Model 實作日誌轉發框架

資料三 112703037 陳凱廷

指導老師：廖竣鋒

1 背景知識

1.1 WebAssembly (WASM)

WebAssembly(WASM) 是一種低階的二進位指令格式, 設計目標是作為多種高階語言的共同編譯目標, 使 C、C++、Rust、Go 等語言均可編譯為同一種可攜式的執行檔案格式。

優點:

- **沙箱隔離:** 每個 WASM 模組擁有獨立的執行環境與線性記憶體空間, 預設無法直接存取宿主的網路、檔案系統或其他系統資源, 有效隔絕外部威脅。
- **啟動快速:** 相較於完整的作業系統程序, WASM 模組的實例化時間極短, 適合作為可熱插拔的插件機制。
- **跨語言:** 不同語言的開發者皆可撰寫 WASM 模組, 無需綁定特定語言生態系。

缺點:

- **記憶體管理繁瑣:** 呼叫者與 WASM 模組之間共用記憶體的方式沒有統一標準, 雙方需自行協商資料佈局, 且須手動處理記憶體邊界問題。
- **複製開銷:** 由於每個模組擁有獨立的線性記憶體, 跨模組的每次資料傳遞都必須進行記憶體複製。
- **缺乏標準介面:** 模組間的序列化 (serialization) 方式沒有統一規範, 不同 WASM 模組若要互相呼叫, 需要雙方事先約定資料格式。

1.2 WebAssembly Component Model (WCM)

WebAssembly Component Model(WCM) 是在核心 WASM 基礎上制定的一套標準規範, 旨在解決模組間互通性的問題。WCM 定義了 **WIT**(WebAssembly Interface Types) 作為統一的介面描述語言, 並透過 **Canonical ABI** 規範高階型別在模組邊界的表示方式; 此外, WCM 亦將網路、檔案、時鐘等系統能力透過 **WASI**(WebAssembly System Interface) 加以標準化, 使宿主得以精確控制每個元件所能使用的能力範圍。

優點:

- **統一介面:** WIT 介面消除了模組間的互通混亂問題, 不同語言撰寫的元件只要實作相同的 WIT 介面, 即可無縫替換。
- **能力模型:** 每個元件預設不具任何系統能力, 須由宿主透過 WIT 與 WASI 顯式授予, 實現以能力 (capability) 為基礎的最小權限隔離。
- **故障隔離:** 單一元件崩潰 (panic 或越界存取) 不會拖垮宿主程序, 影響範圍侷限於該元件的沙箱實例。

缺點:

- **Canonical ABI 複製開銷:** WCM 在元件邊界進行高階型別與線性記憶體間的雙向轉換時, 每次資料進出沙箱均會發生複製, 此成本在多階段管線中會隨階段數累積。
- **生態系尚在成熟中:** WCM 的工具鏈與語言支援相較於核心 WASM 仍較不完善。

2 研究動機與選擇 WCM 的理由

現有的日誌轉發器 (如 Fluent Bit、Fluentd、Vector) 雖各有成熟的插件機制, 但普遍存在兩項根本性的限制:

第一, 語言綁定問題。每套轉發器使用各自的插件語言——Fluent Bit 使用 C 外掛、Fluentd 使用 Ruby 外掛、Vector 使用自行設計的 VRL 領域專用語言。使用者若要客製化處理邏輯, 必須針對每套工具學習對應的語言與 API, 無法重用已有的技術棧。

第二, 安全隔離不足。Fluent Bit 與 Fluentd 的外掛直接編譯或載入進入宿主程序, 完全信任外掛邏輯的行為; 一旦外掛含有漏洞或惡意程式碼, 即可任意存取網路、檔案系統, 甚至導致整個程序崩潰。

為了同時解決這兩個問題, 本研究採用 WASM 沙箱將日誌處理的各個階段封裝為獨立的執行單元。然而, 若僅使用核心 WASM, 模組間的資料傳遞介面沒有統一標準, 開發者仍需手動管理記憶體邊界, 且不同語言的模組難以互通。

因此, 本研究選擇以 **WebAssembly Component Model** 作為框架基礎。WCM 透過 WIT 介面統一了模組間的溝通方式, 使任何能編譯為 WASM 的語言都可以撰寫可替換的處理元件; 其能力模型則讓宿主能精確授予每個元件所需的最小權限, 從根本上防止未授權的系統存取。換言之, WCM 所換取的是: 以一定的 Canonical ABI 轉換開銷, 同時獲得**語言無關性**、**可替換性**與**能力隔離**三項傳統外掛機制難以兼得的特性。

3 研究問題

本研究以日誌轉發 (log forwarding) 為場景, 實作一套以 WCM 為基礎、各階段 (Parse → Filter → Format → Transport) 皆可替換的管線框架, 並圍繞「以效能換取安全與彈性, 其代價是否值得」此一核心, 提出下列研究問題:

3.1 使用 WCM 的效能成本量化與比較

相較於同程序內的原生函式呼叫, 以及僅使用核心 WASM 搭配 C-ABI(offset+length) 的方式, 採用 Component Model 建構日誌管線在吞吐量與延遲上的額外成本為何? 其中有多少比例可歸因於 WASM 沙箱本身, 又有多少來自 Canonical ABI 的邊界型別轉換與複製, 拆解成本來源。

3.2 WCM 為基礎的作法與市面上工具的效能比較

前述比較將本框架對照於原生函式呼叫與裸 core-wasm 等「理論下限」, 目的在於拆解成本來源; 然而這些對照組並非實務上會被部署的方案。為了衡量本框架距離真正可用於生產環境的工具還有多遠, 本研究進一步將其與業界實際採用的日誌處理方案並列比較, 包括 Fluent Bit 搭配 WASM filter(代表真實的 WASM 部署路徑, 但僅走 C-ABI 且功能侷限於 input/filter)、Fluent Bit 的 Lua 外掛, 以及 Vector 的 VRL(代表經過高度優化的原生 DSL 方案)。本問題試圖回答: 在接受一定效能折損的前提下, WCM 所換取的語言無關性與可替換性, 是否使其在功能彈性上足以與這些工具競爭。

3.3 WCM 為基礎的作法與市面上工具的安全性比較

WCM 的能力模型使每個元件預設不具任何系統能力, 須由宿主透過 WIT 與 WASI 顯式授予; 理論上, 這能比 Fluentd Ruby 外掛 (完全信任外部邏輯、於宿主程序內執行) 提供更安全的插入與替換機制。本問題將實測本研究實作的系統: 在執行不可信或含漏洞的第三方元件時, 能否穩定運作, 並在元件嘗試使用未被授予的能力 (如僅具解析權限的 parser 試圖開啟 socket) 時正確地予以阻擋。

4 實作內容

本研究以 Rust 語言搭配與 WCM 整合最完善的 Wasmtime 實作一個 Host, 負責協調各 Component 之間的資料傳遞, 並定義各 Component 須實作的 WIT 介面檔案。日誌處理流程拆解為解析 (parse)、過濾 (filter)、序列化 (format) 與傳輸 (transport) 四個階段, 每個階段皆為可替換的獨立 Component; 其中僅傳輸階段被授予網路能力, 其餘階段皆在不具外部能力的沙箱中執行。為驗證語言無關性, 各階段將分別以 C、C++、C# 與 Go 等不同語言實作對應的 Component。

5 比較指標與比較對象

5.1 比較的指標

本研究的比較指標分為效能與安全兩類。效能指標用以量化採用 WCM 所付出的代價, 並使「為什麼慢」具備可解釋的機制, 而非僅呈現結果數字; 安全指標則用以驗證能力模型所換取的隔離保證。

Table 1: 效能比較指標

指標	單位 / 量法	重要性
吞吐量	records/s、MB/s(穩態)	forwarder 的命脈
延遲	per-batch p50/p95/p99/p999	尾延遲對日誌管線很關鍵
單筆邊界開銷	ns/record(以 identity 元件 microbench)	直接量出 Canonical ABI 的複製成本
CPU 成本	cycles/record(固定輸入速率下)	比 CPU 使用率百分比更可比
記憶體	RSS、每實例 linear memory、隨階段數成長曲線	多實例 / 多階段下的可擴展性
實例化時間	ms/component(冷啟動)	影響熱抽換與水平擴充
複製次數 / bytes	宿主內部插樁計數	使「為什麼慢」具機制性解釋

安全方面, 由於能力模型所提供的是「行為是否在授權範圍內被允許」這類定性保證, 難以化為連續數值, 故以下列面向作為定性比較的依據: 隔離粒度 (能否針對單一階段精確地拒絕網路或檔案能力)、可繞過性 (在威脅模型下是否存在繞過授權的途徑), 以及故障侷限性 (單一元件發生越界存取或 panic 時, 影響範圍是否侷限於其沙箱實例)。此類指標將以「攻擊被擋下」的案例研究作為佐證, 而非以效能曲線呈現。

5.2 比較的對象

比較對象的選取原則為: 每一組對照都應能單獨隔離出一個變因, 使本系統的成本來源得以逐層拆解 (如表 2)。原生 Rust pipeline 為同程序內的函式呼叫, 跨 WASM 邊界的成本為零, 代表吞吐量的上限, 亦即無沙箱成本的天花板; 裸 core-wasm 搭配 C-ABI(offset+length) 隔離出的是 Component Model 與 Canonical ABI 所帶來的「額外」成本, 可區分出使用原生 WASM 與 WCM 之間的差異; 本研究所提出的 Component Model 系統則為受測主體。在實務工具方面, Fluent Bit 搭配 WASM filter 代表真實的 WASM 部署路徑 (惟其功能僅涵蓋 filter/input 且走 C-ABI), 作為現實參考點; Fluent Bit 的 Lua 外掛與 Vector 的原生 VRL 則代表業界實際採用、且經過高度優化的方案, 用以顯示本系統距離生產環境工具的差距。此外, 選做的子行程搭配 pipe/IPC 隔離, 代表以「另開程序」達成隔離所需付出的成本, 用以回答「為何採用 WASM 而非 fork 一個獨立程序」此一設計選擇。

Table 2: 比較對象與其隔離之變因

對照組	隔離出的變因	選取理由
原生 Rust pipeline(同程序內函式呼叫)	跨 WASM 邊界的成本 (= 0)	吞吐量上限、無沙箱成本的天花板
裸 core-wasm + C-ABI(offset+length)	Component Model / Canonical ABI 的「額外」成本	最關鍵的對照——分離 WASM 本身與元件模型的成本
Component Model(本系統)	—	受測主體
Fluent Bit + WASM filter	真實工具的 WASM 路徑	現實參考點 (僅 filter/input、走 C-ABI)
Fluent Bit Lua / Vector VRL(原生)	業界實際採用的高度優化方案	顯示距離生產環境工具的差距